

Relatório do trabalho de Programação Distribuída, Paralela e Concorrente

- O Problema escolhido e necessidade de concorrência/paralelismo
Problema: Contagem de palavras em múltiplos arquivos de texto.
Concorrência: O programa precisa organizar a execução de muitas tarefas (ler cada arquivo, tratar timeouts, fazer retentativas) e combinar resultados parciais. Isso exige um modelo concorrente, mesmo que rodando em um único núcleo.
Paralelismo: Como a operação de contar palavras é independente por arquivo, podemos executá-las simultaneamente em múltiplos núcleos para ganhar desempenho. O programa usa multiprocessing.Pool para distribuir os arquivos entre processos reais.
Conclusão: O problema pede os dois, concorrência para estruturação e paralelismo para aceleração.
- A Decomposição do problema
Divisão por dados: cada worker recebe um arquivo diferente (ou parte, se o arquivo for muito grande) e aplica a mesma função de contagem.
Divisão por tarefas (secundária): o processo principal faz o merge dos resultados (reduce).
Escolha refletida no código: Pool.map distribui automaticamente a lista de arquivos entre os workers.
- O porque que eu usei processos
Optei por processos (multiprocessing) porque:
Contagem de palavras é intensiva em CPU (operações com strings, dicionários). Em Python, threads não executam código Python em paralelo devido ao GIL – apenas processos oferecem paralelismo real.
Processos têm memória isolada, o que simplifica o design: cada

worker gera seu dicionário sem risco de conflito, e o processo principal os combina.

Isolamento é benéfico para tolerância a falhas – um processo que trava não corrompe o estado dos outros.

- Sincronização e seção crítica

Risco de condição de corrida: Se dois workers tentassem atualizar o mesmo dicionário compartilhado (ex: `shared['palavra'] += 1`), as contagens seriam perdidas.

Como evitei: Não compartilhei estado mutável. Cada worker retorna seu dicionário e o principal faz o merge sequencialmente.

Demonstração exigida: No código, incluí a função `exemplo_secao_critica()` que mostra o uso de `Manager().Lock()` para proteger um dicionário compartilhado, essa seria a solução se precisássemos de um contador global.

- Comunicação entre unidades de execução

Mecanismo: Troca de mensagens via `multiprocessing.Pool.map`. Cada worker retorna um dicionário, o processo principal coleta todos os resultados.

Padrão clássico: MapReduce a fase map produz pares (palavra, contagem) em cada worker, e a fase reduce agrupa por palavra e soma.

- Tratamento de falhas

Tipo de falha tratada: arquivo inexistente/corrompido ou tempo de processamento excessivo.

Deteção: exceções capturadas no worker + timeout configurável (usando `apply_async` com `get(timeout)`).

Reação: cada arquivo tem até 2 tentativas (`max_retries`). Se todas falharem, o arquivo é ignorado e o erro é registrado. O resultado final contém apenas os dados bem-sucedidos.

Efeito na consistência: o resultado é consistente (não inclui dados corrompidos) e o programa continua funcionando – não há perda total por causa de uma falha isolada.

- Para executar leia o (Me leia.md)

